



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Game Engine Architectures

Fachhochschule Oberösterreich

C++ Basics

Basics

C++ Pointers

- Do not store a “variable“ but an address (potentially to a variable)
- A pointer “**references**“ another object
- **Pointers**
 - Do not need to be instantiated instantly (like a reference)
 - Can be declared: **type* name**
 - Have a memory address: **&name**
 - Can query the value at the address: ***name**
 - Can allocate memory: **new**
 - Can destroy objects and free memory: **delete**

	Memory Address	Value
	0	
Pointer	1	A100
Pointer	2	A100
	...	
	...	
Value	100	10
	101	

Basics

C++ Pointers

What is the issue here?
The pointer points nowhere (0), so it
cannot be assigned a value

```
int* intPtr;  
cout << intPtr << endl;  
*intPtr = 3;
```

Basics

C++ Pointers

```
int* intPtr;  
cout << intPtr << endl;  
*intPtr = 3;
```

We have to assign a memory address first, then we can work with it

```
intPtr = new int;  
*intPtr = 3;
```

What is the issue with this program?

```
int main()
{
    int z = 0;
    int* pointer;
    pointer = new int;
    *pointer = 5;
    pointer = &z;
    cout << *pointer << endl;
}
```

What is the issue with this program?

We created an integer in memory but dereferenced the pointer.
=> Now, we have allocated space that is no more reachable...

```
int main()
{
    int z = 0;
    int* pointer;
    pointer = new int;
    *pointer = 5;
    pointer = &z;
    cout << *pointer << endl;
}
```

This results in a
memory leak

We create an integer pointer

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        array[i] = i;
    }
    int* pointer = array;
    for (int i = 0; i < howmany; i++)
    {
        cout << pointer[i] << endl;
    }
}
```


We ask a user how large an array should be

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        array[i] = i;
    }
    int* pointer = array;
    for (int i = 0; i < howmany; i++)
    {
        cout << pointer[i] << endl;
    }
}
```

We allocate space for as many integers as given by the user

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        array[i] = i;
    }
    int* pointer = array;
    for (int i = 0; i < howmany; i++)
    {
        cout << pointer[i] << endl;
    }
}
```

We fill the array with numbers from 0 to the desired size

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        array[i] = i;
    }
    int* pointer = array;
    for (int i = 0; i < howmany; i++)
    {
        cout << pointer[i] << endl;
    }
}
```

We iterate through the array and print the elements

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        array[i] = i;
    }
    int* pointer = array;
    for (int i = 0; i < howmany; i++)
    {
        cout << pointer[i] << endl;
    }
}
```

Basics

C++ Pointer Arithmetic

→ As **pointers** are “**just**” **addresses**, we can perform operations on them

```
int* pointer;  
pointer++;
```

→ Here, we increment the pointer by 1, which makes it point to the next memory address

→ The sizeof operator can help us with arithmetics (if needed)

→ Let's see how we can use this with arrays...

Basics

C++ Pointer Arithmetic

```
int main()
{
    int* ptr;
    ptr = new int[2];
    *ptr = 2;
    *(ptr + 1) = 3;
    delete ptr;
}
```

Memory Address	Value
0	
1	
2	
3	
4	
5	
6	

Basics

C++ Pointer Arithmetic

```
int main()
{
    int* ptr;
    ptr = new int[2];
    *ptr = 2;
    *(ptr + 1) = 3;
    delete ptr;
}
```

	Memory Address	Value
ptr	0	-
	1	
	2	
	3	
	4	
	5	
	6	

Basics

C++ Pointer Arithmetic

```
int main()
{
    int* ptr;
    ptr = new int[2];
    *ptr = 2;
    *(ptr + 1) = 3;
    delete ptr;
}
```

	Memory Address	Value
ptr ↓	0	A1
	1	
	2	
	3	
	4	
	5	
	6	

Basics

C++ Pointer Arithmetic

```
int main()
{
    int* ptr;
    ptr = new int[2];
    *ptr = 2;
    *(ptr + 1) = 3;
    delete ptr;
}
```

	Memory Address	Value
ptr ↓	0	A1
	1	2
	2	
	3	
	4	
	5	
	6	

Basics

C++ Pointer Arithmetic

```
int main()
{
    int* ptr;
    ptr = new int[2];
    *ptr = 2;
    *(ptr + 1) = 3;
    delete ptr;
}
```

	Memory Address	Value
ptr ↓	0	A1
	1	2
	2	3
	3	
	4	
	5	
	6	

Basics

C++ Pointer Arithmetic

```
int main()
{
    int* ptr;
    ptr = new int[2];
    *ptr = 2;
    *(ptr + 1) = 3;
    delete ptr;
}
```

	Memory Address	Value
ptr	0	A1
	1	2
	2	3
	3	
	4	
	5	
	6	

This is basically the same program as before...

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        *(array+i) = i;
    }
}
```

Basics

C++ Pointer Arithmetic

But, we do not index directly, we perform pointer arithmetics

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        *(array+i) = i;
    }
}
```

Basics

C++ Pointer Arithmetic

The pointer should use its initial address + the i^{th} next address, and set the value on this address (*) to i

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        *(array+i) = i;
    }
}
```

Basics

C++ Pointer Arithmetic

Which address is the pointer “array”
pointing to?
Still to the start of the array...an address
which we should never lose!

```
int main()
{
    int* array;
    int howmany;
    cout << "how many elements?" << endl;
    cin >> howmany;
    array = new int[howmany];
    for (int i = 0; i < howmany; i++)
    {
        *(array+i) = i;
    }
}
```

Basics

C++ Pointer Arithmetic

Browse to:

<https://onecompiler.com/cpp/3zq5axhgm>

Main.cpp

HGB_Pointers

NEW

CPP

RUN

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int* array;
7     int howmany=5;
8
9     array = new int[howmany];
10    for (int i = 0; i < howmany; i++)
11    {
12        *(array+i) = i;
13    }
14
15    //Task1: go through the array and print its elements
16    //(one per line) without using [];
17    //Task 1.b: without using the variable "howmany" in a loop!
18
19
20 }
```

STDIN

Input for the program (Optional)

Output:

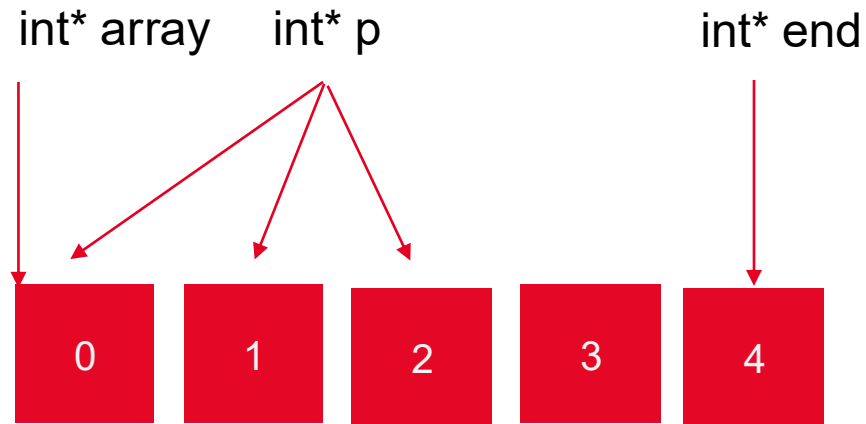
Program did not output anything!

Basics

C++ Pointer Arithmetic

Browse to:

<https://onecompiler.com/cpp/3zq5axhgm>



We create another pointer that points to the end of the array, as well as an “iterator“, a pointer that we can manipulate without losing start or end!

```
int* end = &array[howmany];  
int* iterator = array;  
while (iterator != end)  
{  
    cout << *iterator << endl;  
    iterator++;  
}
```

We use a while loop and print the current value of the iterator pointer before incrementing it

```
int* end = &array[howmany];  
int* iterator = array;  
while (iterator != end)  
{  
    cout << *iterator << endl;  
    iterator++;  
}
```

We end the loop when the iterator has the same value (i.e., the same address) as the end pointer!

```
int* end = &array[howmany];  
int* iterator = array;  
while (iterator != end)  
{  
    cout << *iterator << endl;  
    iterator++;  
}
```

Basics

C++ Pointer Arithmetic

Browse to:

<https://onecompiler.com/cpp/3zqqhwdfw>

Main.cpp

HGB_Pointers_2 

NEW

CPP ▾

RUN ▶

⋮

⌵

```
1 #include <iostream>
2 using namespace std;
3
4 // Implement the concatenate of 2 character arrays using pointer arithmetics
5 char* concatenateStrings(char* string1, char* string2)
6 {
7     return string1;
8 }
9
10 int main()
11 {
12     char s1[50] = "Hello ";
13     char s2[50] = "Hagenberg";
14
15     char* concat = concatenateStrings(s1, s2);
16
17     cout << concat << endl;
18     return 0;
19 }
```

STDIN

Input for the program (Optional)

Output:

Hello

We create two helpers for arithmetic, because we do not want to destroy the original pointers pointing to the start of the strings!

```
char* concatenateStrings(char* string1, char* string2)
{
    char* helper1 = string1;
    char* helper2 = string2;

    while (*(++helper1));
    while (*(helper1++) = *(helper2++));

    return string1;
}
```

We slowly move to the end of the first string, as its null terminated the loop breaks when discovering \0!

```
char* concatenateStrings(char* string1, char* string2)
{
    char* helper1 = string1;
    char* helper2 = string2;

    while (*(++helper1));
    while (*(helper1++) = *(helper2++));

    return string1;
}
```

We assign each following character of the first string the corresponding char value from the second string, until the second one terminates, too.

```
char* concatenateStrings(char* string1, char* string2)
{
    char* helper1 = string1;
    char* helper2 = string2;

    while (*(++helper1));
    while (*(helper1++) = *(helper2++));

    return string1;
}
```


Pre- vs Postincrement: We need to make sure to overwrite only the `\0` of the first string, i.e., at the beginning of the second loop we want to be at the terminator!

```
char* concatenateStrings(char* string1, char* string2)
{
    char* helper1 = string1;
    char* helper2 = string2;

    while (*(++helper1));
    while (*(helper1++) = *(helper2++));

    return string1;
}
```

Game Engine Architectures

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at